

CS242: System Software Lab

System Software Lab

Tut01 : 29th July 2024

Dr. A. Sahu

Dept of Comp. Sc. & Engg.

Indian Institute of Technology Guwahati

Outline

- Course, Attendance, Reference Book
- What do we study in this course?
- Why should this be studied?
- What is “*System Software Lab*” ?
- How is the course structured?

CS433 : Course site, Venue & Timing

- Course website:
<http://jatinga.iitg.ernet.in/~asahu/cs242/>
- Tutorial Class
 - Venue : Core 5, Room- 5101
 - Timing: Mon 8.00AM-9.00AM
- Lab Class
 - Venue: B Tech II Year Lab, CSE, Ground Floor, Ext building
 - Timing: Tue 2.00PM-5.00PM

Course Structure

- Part I (Pre-midsem)
 - Overview of Unix system, commands and utilities;
 - Basic Linux administration and installation: grub, rpm, yum, disk partitioning; Basic Linux utilities, logging, backup, authentication;
 - Program maintenance: make, sccs, git
 - Debugging with gdb and ddd
- Part II (Post-midsem)
 - Archiving: shar, tar; shell use: redirection, .cshrc, environment variables;
 - Regular expression parsing: grep, egrep, sed, awk;
 - Shell programming: bash;

CS242 SysSoftLab: Text/Ref Books

- E. Nemeth, G. Snyder and T. R. Hein, Linux Administration Handbook, Prentice Hall PTR, 2002.
- S. Kochan and P. Wood, Unix Shell programming, 3rd Ed, SAMS, 2003.
- S. Das, Unix System V.4 Concepts and Applications, 3rd Ed, Tata Mcgraw-Hill, 2003.

CS343 : Grading and Rules

- Pre Mid Semester Part (46%)
 - Three Lab Evaluations: (7%+7%+7%)
 - Mid Sem Lab Examination: 25%
- Post Mid Semester Part (54%)
 - Three Lab Evaluations: (8%+8%+8%)
 - End Sem Lab Examination: 30%

Why should this be studied?

- Sys. Soft Lab : After CS101/110
- Why Linux?
 - Open Source, Command based, Driver easy
 - Better Structure
- Useful for designing system
 - Basic Sys Admin:
 - Programming Add: Make, GDB, VCS
 - Using Program useful: Shell, AWK/Sed

What do we study in this course?

- Useful for designing system
 - Basic Sys Admin: installing own software, user management, logging
 - Programming Add: Make, GDB, VCS/GIT
 - Using Program useful: Shell, AWK/Sed for File processing

Course Structure

- Part I (Pre-midsem)
 - A: Overview of Unix system, commands and utilities;
 - Basic Linux administration and installation: grub, rpm, yum, disk partitioning; Basic Linux utilities, logging, backup, authentication;
 - B: Program maintenance: make, sccs, git
 - C: Debugging with gdb and ddd
- Part II (Post-midsem)
 - A: Archiving: shar, tar; shell use: redirection, .cshrc, environment variables;
 - B: Regular expression parsing: grep, egrep, sed, awk;
 - C: Shell programming: bash;

Overview : Part I (A)

- Overview of Unix system, commands and utilities;
- Basic Linux administration and installation: grub, rpm, yum, disk partitioning;
- Basic Linux utilities, logging, backup, authentication;

Basic unix commands that everyone should know

(Even if you have a mac)

What the ~*&?!

- ~ “tilde” indicates your home directory: `/home/you`
- * “star”: wildcard, matches anything
- ? wildcard, matches any one character
- ! History substitution, do not use
- & run a job in the background, or redirect errors
- # % special characters for most crystallography programs
- ` \ ([\ ' back-quote, backslash, etc. special to shell
- _ underscore, use this instead of spaces!!!

Where am I?

pwd

Print name of the “current working directory”

This is the default directory/folder where the shell program will look first for programs, files, etc. It is “where you are” in Unix space.

What is a directory?

`/home/yourname/whatever`

Directories are places you put files. They are represented as words connected by the “/” character. On Windows, they use a “\”, just to be different. On Mac, they are called “folders”. Whatever you do...

DO NOT PUT SPACES

In directory/file names!

What have we here?

ls

List contents of the current working directory

```
ls -l      - long listing, with dates, owners, etc.  
ls -lrt    - above, but sorted by time  
ls -lrt /home/yourname/something  
            - long-list a different directory
```

Go somewhere else?

cd

Change the current working directory

`cd /tmp/yourname/`

- go to your temporary directory

`cd -` - go back to where you just were

`cd` - no arguments, go back “home”
 “home” is where your login starts

A new beginning...

mkdir

Create a new directory.

<code>mkdir ./something</code>	- make it
<code>cd ./something</code>	- go there
<code>ls</code>	- check its is empty

How do I get help?

man

Display the manual for a given program

```
man ls      - see manual for the "ls" command  
man tcsh    - learn about the C shell  
man bash    - learn about that other shell  
man man     - read the manual for the manual
```

to return to the command prompt, type "q"

Move it!

mv

Move or rename a file. If you think about it, these are the same thing.

```
mv stupidname.txt bettername.txt
```

- change name

```
mv stupidplace/file.txt ../betterplace/file.txt
```

- same name, different directory

```
mv stupidname_*.img bettername_*.img
```

Will not work! Never ever do this!

Copy machine

cp

Copy a file. This is just like “mv” except it does not delete the original.

```
cp stupidname.txt bettername.txt
```

- change name, keep original

```
rm stupidname.txt
```

- now this is the same as “mv”

“Permission denied” !?

chmod

Change the “permission” of a file.

```
chmod a+r filename.txt
```

- make it so everyone can read it

```
chmod u+rw filename.txt
```

- make it you can read/write/execute it

```
chmod -R u+rw /some/random/place
```

- make it so you can read/write everything under a directory

Destroy! Destroy!

rm

Remove a file forever. There is no “trash” or “undelete” in unix.

```
rm unwanted_file.txt
```

- delete file with that name

```
rm -f /tmp/yourname/*
```

- forcefully remove everything in your temporary directory.

Will not prompt for confirmation!

less is more

more

Display the contents of a text file, page by page

`more filename.txt` – display contents
`less filename.txt` – many installs now have a replacement for “more” called “less” which has nicer search features.

`head -n 5 filename.txt` – display 5 line contents

to return to the command prompt, type “q”

After the download...

gunzip

File compression and decompression

```
gunzip ~/Downloads/whatever.tar.gz
```

- decompress

```
gzip ~/Downloads/whatever.tar
```

- compress, creates file with .gz extension

Where the %\$#& is it?

find

Search through directories, find files

```
find ./ -name 'important*.txt'
```

- look at everything under current working directory with name starting with “important” and ending in “.txt”

```
find / -name 'important*.txt'
```

- will always find it, but take a very long time!

Did I run out of disk space?

df du

Check how much space is left on disks

`df` - look at space left on all disks

`df .` - look at space left in the current working directory

`du -sk . | sort -g`

- add up space taken up by all files and subdirectories, list biggest hog last

Why so slow?

ps top

Look for programs that may be eating up CPU or memory.

`top` - list processes in order of CPU usage

`jobs` - list jobs running in background of current terminal

`ps -fHu yourname`

- list jobs belonging to your account in order of what
spawned what

Die Die Die!

kill

Stop jobs that are running in the background

```
kill %1      - kill job [1], as listed in “jobs”  
kill 1234    - kill job listed as 1234 by “ps” or “top”  
kill -9 1234 - that was not a suggestion!  
kill -9 -g 1234 – seriously kill that job and the  
                program that launched it
```

Matching a specific pattern in a File

grep

- The **grep** tool searches for lines matching a specific pattern.
- The general form of the **grep** command is:

grep pattern files

- Example:

```
$grep "Jon" *.c
```

```
projaux.c: * Created by: Jon D. Smith
```

```
projaux.c: * Last Modified by: Jon Smith
```

Manual

man

Manual about any command (all the in-out options and usage)

```
$man man
```

```
$man vim
```

```
$man gcc
```

```
$man cp
```

Merging two command

Unix pipe |

- pipe (|) creates a channel from one command to another.
- Think of the pipe as a way of connecting the output from one command to the input of another command.
- Examples:
 - Count the number of users logged onto the current system.
 - The **who** command will give us line by line output of all the current users.
 - We could then use the **wc -l** to count the number of lines...
 - **who | wc -l**

Logging

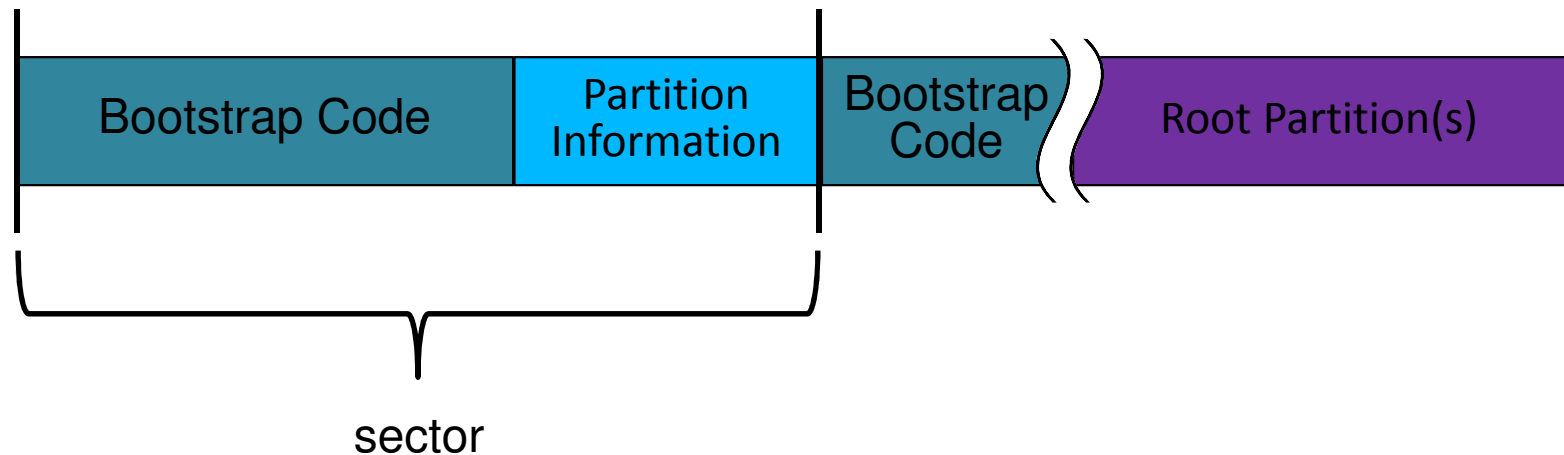
Linux Admin and installation

**grub, rpm/yum, disk partitioning,
and logging;**

grub

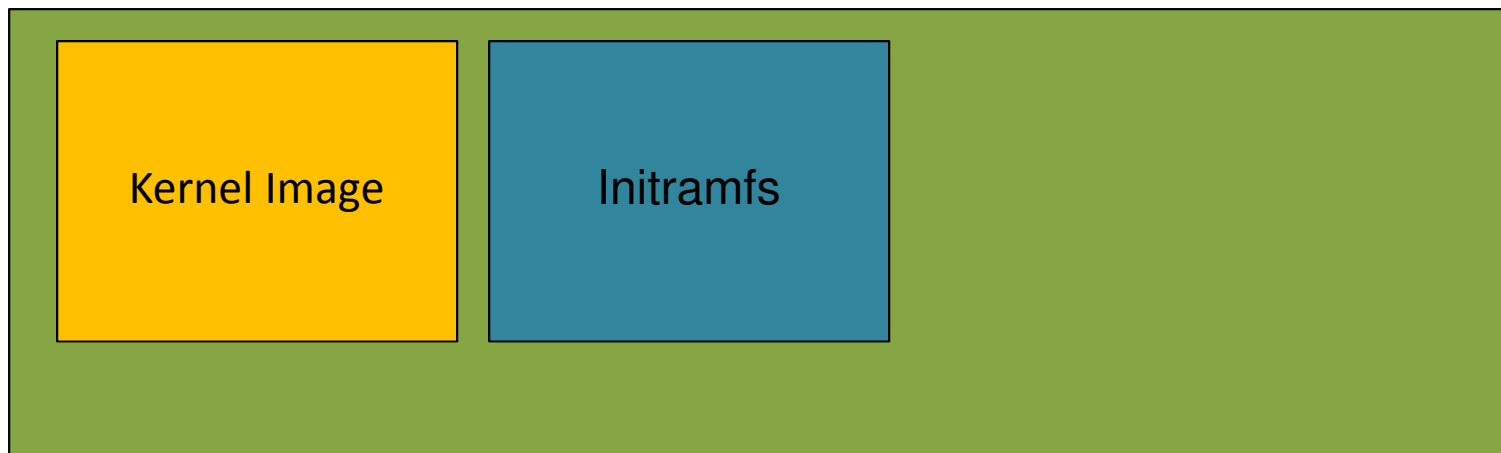
BIOS

- Used to install
 - Multiple OS in a system (Linux and Window)
 - Multiple version of same OS (many kernel of OS)
- Initialize the hardware
- Load the bootstrap code



Bootloader: grub

- Load the kernel
 - Image
 - Initramfs
 - Kernel command line parameters
- E.g., Grub



Linux Admin and installation

**rpm, yum
or
dpkg or apt-get**

Package management : Ubuntu Example

dpkg
apt-get

Debian package naming scheme:

package_version-build_architecture.deb

- **package:** name of the application being installed.
- **version:** version number of the application.
- **build:** build number of the package. Each time the package is redone this number is incremented.
- **architecture:** platform the package was compiled for.

There's a special type of package known as a ***task package***.

These packages are empty packages without software, but with dependencies. Used to install a large “task” on the system.

Using dpkg

Installing packages: ***dpkg -i package_file.deb***

```
debian:~# dpkg -i ethereal_0.8.13-2_i386.deb
```

```
Selecting previously deselected package ethereal.
```

```
(Reading database ... 54478 files and directories currently  
installed.)
```

```
Unpacking ethereal (from ethereal_0.8.13-2_i386.deb) ...
```

```
dpkg: dependency problems prevent configuration of ethereal:
```

```
ethereal depends on libpcap0 (>= 0.4-1); however:
```

```
Package libpcap0 is not installed.
```

```
dpkg: error processing ethereal (--install):
```

```
dependency problems - leaving unconfigured
```

```
Errors were encountered while processing:
```

```
Ethereal
```

Using dpkg

Removing packages: ***dpkg --remove package_name.deb*** or ***dpkg -r package_name.deb***

```
debian:~# dpkg --remove libpcap0
dpkg: dependency problems prevent removal of libpcap0:
ethereal depends on libpcap0 (>= 0.4-1).
dpkg: error processing libpcap0 (--remove):
dependency problems - not removing
Errors were encountered while processing:
libpcap0
```


Using apt-get

installing packages: *apt-get install package_name*

```
debian:~# apt-get install nano
```

```
sofiya@sofiya-VirtualBox:~$ sudo apt-get install nano
Reading package lists... Done
Building dependency tree
Reading state information... Done
Suggested packages:
  hunspell
The following NEW packages will be installed:
  nano
0 upgraded, 1 newly installed, 0 to remove and 241 not upgraded.
Need to get 269 kB of archives.
```

disk partitioning

Using fdisk

```
debian:~# fdisk -l
debian:~# fdisk /dev/sdb2
m -> help
p -> print partition table
n -> create new partition
d -> delete partition
q -> quit without writing
w -> write to disk
```

Using fdisk

- On a disk, we can have a maximum of four partitions.
- Partitions are of two types.
 - Primary
 - Extended
- Extended partitions can have logical partitions inside it.
- Among the four possible partitions, the possible combinations are.
 - All 4 primary partitions
 - 3 primary partitions and 1 extended partition

Using fdisk

```
debian:~# mkfs.ext4 -j /dev/sdb1
```

Create ext4 FS in sdb1

```
debian:~# mkfs.fat /dev/sdb2
```

Create vfat in sdb1

Using gparted

```
debian:~# apt-get install gparted
```

GUI version of fdisk

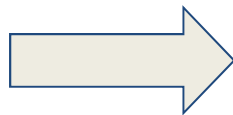
More versatile

A document will be uploaded on gparted

Logging

- A Log file is a file that records events occurs in a operating system or other system software or the messages between the different user on a communication software.
- Logging is a act of keeping log files.

Significance



System admin need to know what is happening on the system.

Logging

- A general purpose facility called syslog is used.
- Programme send their log entry to syslog by consulting two configuration files **/etc/syslogd.conf** and **/etc/syslog**.
- Syslog contain four basic terms:
 - **Facility:** Describe the type of application. For example: mail, kernel, ftp etc.
 - **Priority or Levels:** Describe the importance of log message. For example: emerg, crit, alert etc.
 - **Selector:** Combination of facility and levels.
 - **Action:** Whenever there is a match of selector, a action is performed eg: message to log file, echo the message to console.

Logging (Contd.)

- The syslog.conf controls where message are logged. A typical syslog.conf look like this:

```
*.err;kern.debug;auth.notice /dev/console
daemon,auth.notice          /var/log/messages
lpr.info                     /var/log/lpr.log
mail.*                       /var/log/mail.log
ftp.*                         /var/log/ftp.log
auth.*                       @prep.ai.mit.edu
auth.*                       root,amrood
netinfo.err                  /var/log/netinfo.log
install.*                    /var/log/install.log
*.emerg                       *
*.alert                      |program_name
mark.*                       /dev/console
```

Group Management

- Three type of account in the linux system:
 - **Root account:** Have complete control of system.
 - **System account:** Have some system specific control. For example: sshd account and mail account.
 - **User account:** Have interactive access to the system and provide very limited access to critical data.
- To manage user and groups, four type of administration files are needed:
 - /etc/passwd - Keep user account and password information.
 - /etc/shadow - Hold encrypted password.
 - /etc/group - contain group information.
 - /etc/gshadow - Hold secure information related to groups.

Thanks